ALGORITHMS

Patient Data Management System Using Doubly Linked List

Algorithm 1: Main Algorithm

1. Start.
2. Initialize **head = NULL**.
3. Display the main menu with options to add, display, search, delete, display forward/reverse, and exit.
4. Accept the user's choice.
5. Call the function corresponding to the selected operation.
6. After completing the operation, return to the main menu.
7. If Exit is selected, release allocated memory and terminate the program.
8. Stop.

Algorithm 2: Add Patient

1. Start.
2. Enter Patient ID, name, age, and disease.
3. Create a new node dynamically.
4. Store the patient details in the new node.
5. Set the new node's **prev** and **next** pointers appropriately.
6. If the list is empty, set **head** to the new node.
7. Otherwise, traverse to the last node, link its **next** to the new node, and set the new node's **prev** to the last node.
8. Display a successful insertion message.
9. Return to the main menu.

Algorithm 3: Display Patient Records (Forward)

1. Start.
2. If **head == NULL**, display “No patient records found” and return.
3. Set **temp = head**.
4. Display the Patient ID, name, age, and disease stored in the current node.
5. Move to the next node using **temp = temp->next**.
6. Repeat steps 4–5 until **temp == NULL**.
7. Return to the main menu.

Algorithm 4: Display Patient Records (Reverse)

1. Start.
2. If **head == NULL**, display “No patient records found” and return.
3. Set **temp = head** and traverse to the last node using the **next** pointer.

4. Display the patient details stored in the current node.
5. Move backward using **temp = temp->prev**.
6. Repeat steps 4–5 until **temp == NULL**.
7. Return to the main menu.

Algorithm 5: Search Patient

1. Start.
2. Enter the Patient ID to search for.
3. Set **temp = head**.
4. Compare the Patient ID with the ID in the current node.
5. If they match, display the patient's details and stop searching.
6. Otherwise, move to the next node and repeat the comparison.
7. If **temp == NULL**, display “Patient not found.”
8. Return to the main menu.

Algorithm 6: Delete Patient

1. Start.
2. Enter the Patient ID to delete.
3. Set **temp = head** and search for the matching node.
4. If no matching node is found, display “Patient not found.”
5. If the node is the first node, update **head** to the next node.
6. If a previous node exists, set its **next** pointer to the node after the target.
7. If a next node exists, set its **prev** pointer to the node before the target.
8. Release the target node's memory.
9. Display a successful deletion message.
10. Return to the main menu.

Algorithm 7: Exit

1. Start.
2. Traverse the list and release the memory of each node safely.
3. Set **head = NULL**.
4. Display the exit message.
5. Stop.

Note: Use the same operation names and Patient ID field as in your actual C++ program.